CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY

Department of Computer Science and Engineering

Operating Systems

CASE STUDY 14

Race Condition in XV6 Shared Data

Study of shared physical-memory free-list synchronization

Team members

Name	Roll number
L Nithish Kumar Goud	160124733327
G Ishan	160124733322
Santhosh	160124733332
Venkat	160124733340

Submitted to: _____

Faculty / designation: _____

Semester / academic year: _____

Submission date: _____

Certificate and team responsibility

Team name: Case Study 14 — XV6 Shared Data

Problem statement: Study race conditions when multiple processes or CPUs access shared kernel data without synchronization, and analyze how xv6 locks prevent inconsistent updates.

Table 1. Supplied four-member responsibility allocation

Member	Lead responsibility in division plan	Share
L Nithish Kumar Goud (160124733327)	Technical implementation, experiment, evidence and results	25%
G Ishan (160124733322)	Lock-removal analysis and race timelines	25%
Santhosh (160124733332)	Spinlock and critical-section explanation	25%
Venkat (160124733340)	Foundations, shared-data inventory and report integration	25%

Faculty name: _____ Signature: _____ Date: _____

Index

Table 2. Report contents

Section	Page
Case statement, objectives, activities and expected outcome	4
1. What is a race condition?	4
2. Problems generated by race conditions	5
Existing system and background	5
3. Proposed solution	6
4. What is synchronization?	6
5. Types of synchronization	6
6. How xv6 protects shared data	7
Working flow	8
7. C files, WSL commands and experiment sequence	8
7.6. Kernel allocator critical sections	12
7.7. Spinlock ownership and memory ordering	13
7.8. Guest workload gate and checker	14
7.9. Host runner phases and restoration	15
8. What the locks prevent, and possible unlocked outcomes	15
9. Comparison of results	16
10. Conclusion, applications and future scope	19
References	20

Case Study 14: Problem Statement, Objectives, Activities and Expected Outcome

Problem Statement: Study how race conditions can occur when multiple processes or CPUs access shared kernel data without proper synchronization. Analyse how XV6 prevents inconsistent updates using locking mechanisms.

The case uses the physical-memory free list as the shared kernel data structure. The study follows the access paths and the lock associated with the structure, reasons about possible outcomes when the lock calls are removed, and compares locked, unlocked, and restored behavior using the same user workload.

Objectives

The objectives are to understand race conditions and critical sections; identify shared kernel data structures in xv6; study the need for mutual exclusion; observe the effect of concurrent updates; and explain how locks prevent inconsistent results.

Activities

The completed source review identifies the allocator free list and its lock. The experiment used four CPUs and a coordinated multi-process workload. It compared original locking, a temporary removal of four allocator lock calls, and restoration. The report distinguishes possible source-level interleavings from events directly proven by the logs.

Expected outcome

Students should be able to explain how race conditions arise and how synchronization mechanisms provide safe access to shared kernel data.

1. What is a race condition?

> **Race condition**: concurrent operations access shared data, and the result depends on how their steps interleave.

A critical section reads or changes shared state; mutual exclusion lets only one participant execute that section at a time.

The motivation is to connect synchronization theory to a real kernel resource. xv6 manages memory on multiple CPUs, so a list update that appears simple in source can become unsafe when operations overlap.

The xv6 race is a multi-step pointer update. kalloc() reads the free-list head P, obtains P->next, advances kmem.freelist, and returns P. If CPU 1 reads the same P before CPU 0 advances the head, both calls can return the same physical page. Each process can then write through a different virtual address that maps to that one page.

Table 3. Possible duplicate allocation without kmem.lock

Step	CPU 0: kalloc()	CPU 1: kalloc()	Shared head
1	r0 = kmem.freelist = P		P
2		r1 = kmem.freelist = P	P

Step	CPU 0: kalloc()	CPU 1: kalloc()	Shared head
3	kmem.freelist = P->next		P->next
4		kmem.freelist = P->next	P->next
5	return P	return P	Both callers own P

A second dangerous path occurs in kfree(): two CPUs can each link a returned page to the same stale head, then overwrite one another's head update. The resulting schedule may lose a page or break the list. These are source-level possibilities; the logs do not reconstruct which instructions interleaved in a given attempt.

2. Problems generated by race conditions

When operations that preserve a shared-data invariant interleave without the required lock, one update can overwrite another or a reader can observe a partly updated structure. In the page allocator, possible consequences include returning the same page twice, losing a free page, corrupting a next pointer, failing an allocation, or trapping in the kernel.

Table 4. Possible problems and their effect

Problem	Possible allocator cause	Possible effect
Lost update	One CPU overwrites a newer free-list head	A page becomes unreachable from the list
Duplicate allocation	Two CPUs read the same head before either removes it	Two callers may use the same physical page
Pointer or list corruption	Conflicting writes damage a next pointer or head	Later allocator access may fail or trap
Allocation/test failure	Allocator returns bad state or cannot satisfy request	A process reports failure or kernel activity stops
Deadlock confusion	A run does not finish within a timeout	Timeout alone does not show why it stopped

A race is a correctness problem. A deadlock is a progress problem in which participants wait indefinitely for conditions that cannot be met. The experiment records a host timeout as TIMEOUT, not as a diagnosed deadlock. A panic or trap also does not prove which exact instruction interleaving occurred.

Existing System and Background

Standard xv6 uses one global kmem.lock to protect its physical-page free list. This coarse-grained design is simple and preserves list consistency, but every CPU requesting or freeing a page shares the same lock and may spin while another CPU owns it. The MIT xv6 locking chapter discusses this contention and per-CPU free lists as a possible design that reduces sharing, at the cost of refill or page-stealing logic.

A lock-free compare-and-swap (CAS) loop is another design option. It avoids waiting on one lock but needs careful handling of pointer lifetime and ABA, where a location changes from A to B and back to A so a CAS cannot detect the intervening change. Per-CPU lists and lock-

free allocation were background alternatives only; neither was implemented or benchmarked here.

Other shared xv6 data reviewed at source level: process state (p->lock), PID allocation (pid_lock), parent/child wait state (wait_lock), buffer-cache metadata (bcache.lock), and timer ticks (tickslock). These structures were not experimentally modified.

3. Proposed Solution: protect the complete update

A shared invariant needs one consistent synchronization rule. For the allocator, every operation that reads or changes the free-list head and links must use kmem.lock. The lock must cover dependent reads as well as writes; protecting only the final assignment would leave the same stale-read race.

A spinlock provides mutual exclusion by giving one CPU ownership of the critical section while other CPUs wait. Atomic exchange provides the cross-CPU ownership test. Acquire ordering ensures following protected operations occur after acquisition, while release ordering publishes the completed updates before another CPU acquires the lock. xv6 also disables local interrupts while holding a spinlock to avoid same-CPU interrupt re-entry. Interrupt disabling by itself cannot exclude a second CPU.

The tested source already contains this solution. The experiment removes the acquire and release calls temporarily, keeps the workload and list operations unchanged, then restores the original allocator byte for byte and rebuilds it. The experiment therefore demonstrates an existing xv6 mechanism rather than introducing a new one.

4. What is synchronization?

> **Synchronization**: coordination of concurrent operations so shared state remains correct.

Mutual exclusion is one form. Other mechanisms coordinate ordering, wait for a condition, count available resources, or mark a common phase boundary.

Table 5. Spinlock operation in the checked xv6 revision

Stage	Operation	Purpose
Before acquire	push_off()	Disable local interrupts and track nesting
Acquire	Atomic exchange with acquire ordering	Claim exclusive ownership across CPUs
Critical section	Update kmem.freelist and page links	Preserve the allocator invariant
Release	Atomic store with release ordering	Publish updates and make the lock available
After release	pop_off()	Restore prior interrupt state when nesting reaches zero

In kernel/spinlock.c, acquire uses __atomic_exchange_n with __ATOMIC_ACQUIRE and release uses __atomic_store_n with __ATOMIC_RELEASE. push_off and pop_off manage local interrupt state and nesting. The atomic exchange supplies cross-CPU mutual exclusion. Local interrupt disabling alone does not provide it.

5. Types of synchronization

Table 6. Common synchronization mechanisms

Mechanism	How it coordinates work	Typical use or limitation
Mutex	One thread owns a protected region at a time	Blocking mutual exclusion in thread libraries
Spinlock	One CPU owns the region; others wait actively	Short kernel critical sections; consumes CPU while waiting
Semaphore	A counter tracks available permits	Bounded resources or event counts
Condition variable / monitor	A thread sleeps until a predicate can be checked again under a mutex	Wait for a state change without busy waiting
Reader-writer lock	Many readers or one writer	Read-heavy data; writer fairness and complexity matter
Barrier	Participants wait at a phase boundary	Parallel computation phases
Atomic operation	One indivisible machine update	Single values; not automatically a whole-object invariant

The project measured only xv6's physical allocator spinlock. The other mechanisms are background concepts and were not implemented or benchmarked in this study.

6. How xv6 protects the shared data

The checked source revision is f780d7850765d74cbdcd1fff4b822d8829f7c9bd. In kernel/kalloc.c, kmem.freelist points to unused physical pages and kmem.lock protects updates to the list. kalloc removes the first page while holding the lock. kfree links a released page at the head while holding the same lock.

Table 7. xv6 files examined and their role

File	Role in the path	Status in this study
kernel/kalloc.c	Defines kmem.freelist and kmem.lock; implements kalloc and kfree	Lock calls removed and restored for the measured experiment
kernel/spinlock.c / spinlock.h	Implements initlock, acquire, release, holding, push_off, pop_off	Read to explain the existing lock
user/race_test.c	Runs the four-child memory verification workload inside xv6	Included in source archive and unchanged across phases
kernel/sysproc.c	sys_sbrk dispatches memory growth or shrink	Source-level path review
kernel/proc.c	growproc manages process address-space size	Source-level path review
kernel/vm.c	uvmalloc/uvmdealloc and uvmunmap allocate or release pages	Source-level path review
user/ulib.c	sbrk wrapper invokes the system call	Source-level path review
member1_tools.c	Host-side C runner builds, runs, records, restores, packages	C implementation used for fresh validation

The path for growth is user/ulib.c sbrk, kernel/sysproc.c sys_sbrk, kernel/proc.c growproc, kernel/vm.c uvmalloc, then kernel/kalloc.c kalloc. Shrink follows growproc to uvmdealloc

and uvmunmap, then kfree. Fork and exit also exercise the allocator, but the stress program's direct path is through sbrk.

Working flow

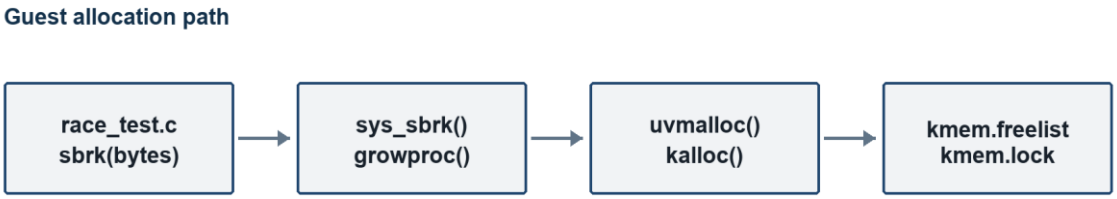


Figure 1. Guest allocation and release path from sbrk() to the shared allocator.

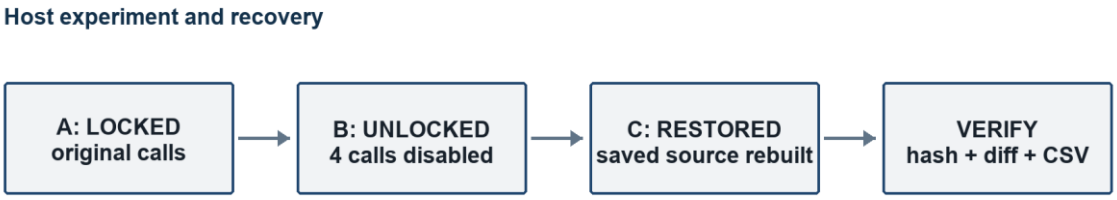


Figure 2. Locked, unlocked, and restored experiment phases with final checks.

Each attempt boots a fresh QEMU guest. The host runner captures console output, classifies one primary result, and appends the run ID and evidence path to results.csv.

7. C files, WSL commands and experiment sequence

The case study has two project C programs with different execution environments. user/race_test.c is a guest user program compiled by the xv6 RISC-V Makefile. member1_tools.c is a host program compiled by Ubuntu cc under WSL. kernel/kalloc.c is xv6 kernel C and contains the existing allocator. The lock-removal variant changes only four acquire/release calls in that source file.

7.1 user/race_test.c: guest workload

The parent creates a pipe, forks four children, and waits until all children exist before releasing them. Each child repeats 200 rounds. In one round it requests 32768 bytes through sbrk, fills the allocation with its distinct byte tag, waits briefly in user space, and checks every byte. It then shrinks the allocation using sbrk(-32768). The parent waits for all children and checks setup and exit statuses before printing PASS or FAIL.

The pipe gate encourages concurrent activity. Each request covers eight 4096-byte pages, so all four children repeatedly exercise the kernel allocator. The test checks the contents visible in each user buffer. It may miss a page lost from the free list or corruption outside those buffers. volatile forces verification reads in this program; it is not a lock and does not synchronize kernel access.

7.2 member1_tools.c: host runner

The C host helper automates builds, QEMU runs, timed console capture, experiment phases, restoration, and result packaging. It preserves console logs and terminal recordings, writes per-attempt CSV rows, checks the restored allocator, and can replay a saved transcript. The following table maps every named helper in the source to its role.

Table 8. member1_tools.c function map

Function	Responsibility
signal_stop	Records a stop request so the runner can shut down and restore during handled interruption
fail	Reports a fatal error and terminates the current operation
path_join	Builds a checked path from directory and filename parts
load / save	Reads and writes complete files used for snapshots, logs and generated summaries
execute	Starts a child process, captures output, enforces time limits, and reports status
checked	Runs a required command and fails if it returns an unsuccessful status
restore	Copies the saved allocator back and rebuilds the restored kernel
json_string	Escapes text for JSON output
csv_text	Quotes and escapes CSV text fields
message_line	Appends one output line to a captured transcript
build	Runs and records the xv6 build commands
record_command	Saves the command and its output for the evidence archive
run	Coordinates preflight, locked attempts, lock removal, unlocked attempts, restoration, and restored attempts
csv_row	Writes one attempt record with phase, status, elapsed time, and evidence filenames
compare_names	Sorts file names for stable packaging and manifest generation
manifest_tree	Walks the evidence tree and creates file/checksum records
package	Checks the results, writes summaries/checksums, and creates the handoff archive
main	Parses the run, package, and replay command-line modes

A handled Ctrl+C requests the runner shutdown path, which restores and rebuilds the allocator. A power loss or forced termination such as SIGKILL cannot run that cleanup. The saved final restoration check and empty allocator diff are the evidence that the checked-in working source was restored.

7.3 Environment and fixed workload

Table 9. Recorded experiment configuration

Item	Recorded value
Source	xv6-riscv commit f780d7850765d74cbdcd1fff4b822d8829f7c9bd
Host and guest	Ubuntu 24.04 under WSL2; RISC-V xv6 guest
Emulator and compiler	QEMU 8.2.2; RISC-V GCC 13.3.0

Item	Recorded value
Guest resources	4 CPUs; 128 MiB RAM
Workload	4 children; 8 pages per child allocation; 200 rounds
Attempts per phase	10 locked; 10 unlocked; 10 restored
Run isolation	Fresh QEMU guest for each attempt
Limits	30 seconds to reach shell; 60 seconds after stress command
Instrumentation	No artificial kernel delay; no kernel timing instrumentation

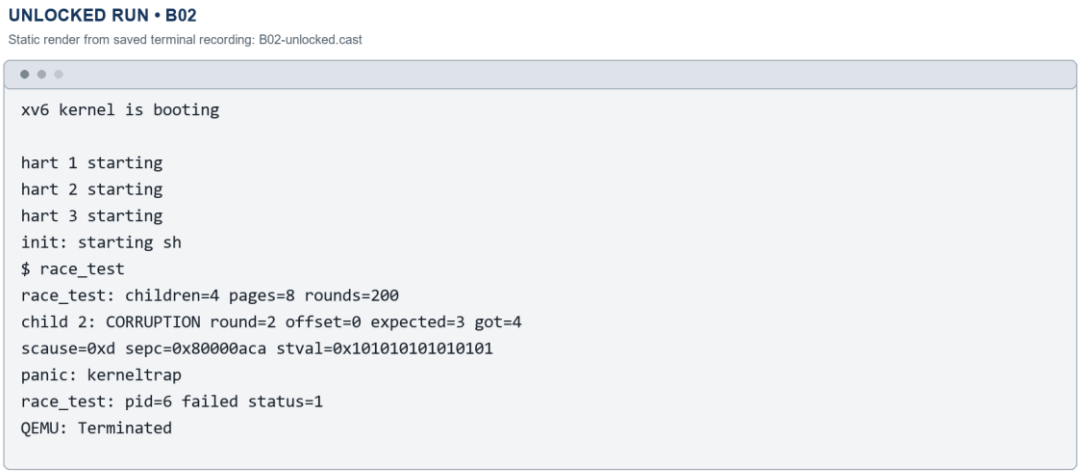


Figure 3. Static terminal still reconstructed from saved B02 recording; output includes corruption and kerneltrap.

7.4 WSL setup and command sequence

On this PC, Ubuntu 24.04, QEMU, and the RISC-V toolchain were already available. The following PowerShell command starts the distribution. The package-install commands are included for a new Ubuntu installation and were not part of the measured attempts.

```
# PowerShell
wsl -d Ubuntu-24.04
# New Ubuntu setup only
sudo apt update
sudo apt install git build-essential qemu-system-misc gcc-riscv64-linux-gnu binutils-
riscv64-linux-gnu bc
```

For a new checkout, use an unused directory. Do not clone over the completed ~/xv6-member1 checkout. The revision below is the fixed commit used by the case study.

```
git clone https://github.com/mit-pdos/xv6-riscv.git ~/xv6-member1
cd ~/xv6-member1
git checkout --detach f780d7850765d74cbdcd1fff4b822d8829f7c9bd
git switch -c member1-race-study
```

The completed checkout includes user/race_test.c and a UPROGS entry in Makefile. The entry uses a literal tab before the target.

```
# Add to the existing UPROGS list in Makefile
    $U/_race_test\
```

The ordinary build and sanity sequence boots xv6, checks the known revision, runs the saved OS sanity command, and then invokes the stress program from the xv6 shell.

```
cd ~/xv6-member1
git rev-parse HEAD
make -j4 kernel/kernel fs.img
make qemu CPUS=4
# At the xv6 shell:
echo hello xv6
usertests -q
race_test
# Exit QEMU with Ctrl+A, then X
```

The growth path begins at sbrk and ends at kalloc. Shrinking follows uvmdealloc and uvmunmap to kfree. The test is run as a user program and does not call kernel functions directly.

7.5 C runner commands and the three measured phases

The host runner is compiled with the Ubuntu C compiler. The following run command is a reproducible fresh-run example that writes to a new evidence directory. It is not presented as the exact historical output path. The runner refuses a directory that already exists and expects the original allocator to match Git HEAD.

```
cd '/mnt/c/Users/HP/Documents/OS record/OS CASE STUDY'
cc -std=c11 -Wall -Wextra -Werror -O2 member1_tools.c -o /tmp/member1_tools
/tmp/member1_tools run /home/nithish/xv6-member1 /home/nithish/member1-new-evidence
full
```

Run mode performs sanity and locked smoke checks before the measured attempts. It then executes the same workload across the three phases. A fresh guest boots for every attempt.

Table 10. Phase sequence handled by the C runner

Phase	Allocator condition	Recorded run IDs	Restoration action
A: locked	Original kmem.lock calls present	A01–A10	Save source and establish baseline
B: unlocked	Comment out acquire/release in kalloc and kfree	B01–B10	Keep allocator data operations and workload unchanged
C: restored	Original allocator restored byte for byte	C01–C10	Verify checksum, empty final diff, rebuild and repeat

The runner has separate package and replay modes. Replay prints a saved transcript, while package validates the archived CSV, builds summaries and checksums, and creates the compressed handoff.

7.6 Kernel allocator critical sections

The experiment targets the linked list in kernel/kalloc.c. kmem.freelist points to the first free physical page, and each free page stores its next pointer in struct run. A page can be removed or inserted only while the list head and the dependent next pointer are protected by the same lock.

```
/* kalloc(): detach one page from the list */
acquire(&kmem.lock);
r = kmem.freelist;
if (r)
  kmem.freelist = r->next;
release(&kmem.lock);

/* kfree(): prepend one page to the list */
acquire(&kmem.lock);
r->next = kmem.freelist;
kmem.freelist = r;
release(&kmem.lock);
```

The lock protects the transfer between the shared list and a caller. kalloc() fills the removed page after releasing the lock because the page is no longer reachable by another allocator. kfree() fills the page before linking it, so no other allocator can obtain that page during the fill. The critical section protects list membership, not later writes made by the process that owns an allocated page.

7.7 Spinlock ownership and memory ordering

The checked kernel/spinlock.c implementation combines local interrupt control with an atomic cross-CPU ownership operation. The atomic exchange is the point where one CPU claims the lock. Acquire ordering keeps protected memory operations after that claim; release ordering publishes completed writes before another CPU obtains the lock.

```
push_off();
if (holding(lk))
  panic("acquire");
while (__atomic_exchange_n(&lk->locked, 1, __ATOMIC_ACQUIRE) != 0)
  ;
lk->cpu = mycpu();
...
__atomic_store_n(&lk->locked, 0, __ATOMIC_RELEASE);
pop_off();
```

Table 11. What each spinlock operation protects

Operation	Effect in this xv6 revision	Boundary
push_off()	Disables local interrupts and tracks nesting	Does not exclude another CPU
Atomic exchange	Changes unlocked to owned as one operation	Other CPUs spin while owned
Acquire ordering	Keeps critical-section accesses after ownership	Does not choose the data invariant
Release store	Makes protected updates visible before unlock	Does not guarantee fairness
pop_off()	Restores interrupt state when nesting returns to zero	Runs after ownership is released

holding() checks whether the current CPU owns the lock, while cpu and locked fields support ownership checks and diagnostics. The lock is intentionally short: the allocator releases it before page-filling work in kalloc(). Removing acquire/release also removes the interrupt-handling effects of those calls when no outer spinlock is held, so the measured change is not an isolated test of atomic ownership alone.

7.8 Guest workload gate and memory checker

user/race_test.c creates four children and holds each child at a pipe read until all forks succeed. The parent then writes four tokens. This narrows the start-time spread without injecting a delay into kernel code. Each worker uses an eight-page allocation, writes a distinct byte tag, waits briefly in user space, reads every byte again, and shrinks the address space.

```
int pid = fork();
if (pid == 0) {
  int received = read(gate[0], &token, 1);
  if (received != 1)
    exit(2);
  exit(worker(id));
}
...
write(gate[1], tokens, NCHILD);

char *allocation = sbrk(bytes);
volatile uchar *memory = (volatile uchar *)allocation;
for (int offset = 0; offset < bytes; offset++)
  memory[offset] = tag;
/* second pass reports a mismatch before sbrk(-bytes) */
```

The explicit user-page workload contributes a lower bound of 4 children × 200 rounds × 8 pages = 6,400 page allocations plus 6,400 releases, or 12,800 kalloc()/kfree() calls if every round completes. This is not a measured count or a total: page-table pages, fork, pipes, and other kernel work add calls. The test checks whether processes retain distinct data in their user buffers; passing does not detect every lost or duplicate page.

The parent treats pipe or fork setup errors and every non-zero wait() status as failure. It reports PASS only when all four children start and exit successfully. volatile forces the verification loop to read memory again; it is not an inter-process barrier and does not make the kernel allocator safe.

7.9 Host runner phases and restoration

member1_tools.c is a host C program compiled inside Ubuntu under WSL. Its run mode saves the original allocator, verifies the fixed revision and workload files, builds xv6, launches QEMU through pipes, polls console output, applies time limits, and records a primary outcome with the raw transcript. The runner starts one fresh guest per attempt and records the two evidence datasets in separate directories.

```
must_restore = 1;
atexit(restore);
...
if (edits != 4)
  exit(1);
save(allocator, variant, dest);
...
else if (phase == 2)
  save(allocator, original, original_size);
...
restore();
sha256sum -c allocator-original.sha256
git diff --exit-code HEAD -- kernel/kalloc.c
```

Before the unlocked phase, the runner comments exactly four acquire/release calls in kalloc.c and saves the resulting diff. Before the restored phase, it writes back the saved allocator bytes and rebuilds kernel/kernel and fs.img. It verifies the saved source checksum and requires an empty final diff. atexit(restore) covers handled process termination; it cannot recover the file after power loss or forced termination. A final verification therefore remains part of the recorded workflow.

For each QEMU run, the host captures console text to a .txt transcript and a timed terminal recording, then appends the attempt ID, phase, status, elapsed time, and evidence filename to results.csv. Elapsed seconds begin when the host recognizes the xv6 shell prompt immediately before sending race_test; they include result detection and a short output-collection window, not boot time or pure guest execution time.

The C-runner selects one primary result from accumulated output in this order: CORRUPTION, PANIC_OR_TRAP, TEST_FAILURE, then PASS. If the host time limit expires, TIMEOUT overrides the detected text. Thus B02 remains CORRUPTION despite a later kerneltrap, while B10 is TIMEOUT despite earlier trap diagnostics. Neither classification establishes the precise instruction interleaving.

From the Windows-mounted project directory, the helper can replay a saved attempt and package the evidence archive:

```
cd '/mnt/c/Users/HP/Documents/OS record/OS CASE STUDY'
/tmp/member1_tools replay member1_completed/evidence/B02-unlocked.txt
/tmp/member1_tools package member1_completed
```

The original results remain in member1_completed/evidence/ and the independent C-runner set in member1_completed/c_validation/. The two 30-attempt datasets are not merged.

8. What the locks prevent, and possible unlocked outcomes

A lock does not prevent concurrency; it makes the protected operations run one at a time with respect to the same lock. That ordering preserves the free-list invariant. Without the lock, a harmful schedule is possible even if the two individual pointer assignments are each machine-word operations.

Table 12. Possible kalloc interleaving without kmem.lock

Step	CPU 0	CPU 1	Possible state
1	Read freelist head P		Head is P
2		Read freelist head P	Both hold stale P
3	Set head to P.next		Head advances
4		Set head to P.next	Head remains advanced
5	Return P	Return P	Both callers may receive P

Table 13. Possible kfree lost-page schedule without kmem.lock

Step	CPU 0 frees A	CPU 1 frees B	Possible state
1	Read old head F		Head is F
2		Read old head F	Both read F
3	Set A.next to F		A points to F
4		Set B.next to F	B points to F
5	Set head to A		Head becomes A
6		Set head to B	A may be lost from reachable list

These schedules are source-level explanations, not captured instruction traces. The recorded corruption, test failures, panics, and traps demonstrate that removing allocator locking produced unsafe outcomes under the tested workload. They do not identify which exact schedule caused each run.

9. Comparison of results

Two datasets are preserved. The original run set was recorded before the host helper was converted to C and is stored under member1_completed/evidence/. The fresh validation run set was recorded by member1_tools.c and is stored under member1_completed/c_validation/. Each dataset has 30 attempts, with 10 attempts in each phase. Counts below are not merged.

Table 14. Outcomes by independent dataset

Dataset	Locked phase	Unlocked phase	Restored phase
Original observations	10/10 PASS	3 corruption; 4 panic/trap; 2 test failure; 1 timeout	10/10 PASS

Dataset	Locked phase	Unlocked phase	Restored phase
C-runner validation	10/10 PASS	1 corruption; 6 panic/trap; 2 test failure; 1 timeout	10/10 PASS

In both datasets, every attempt reached the xv6 shell and started race_test. The locked and restored phases passed in all ten attempts. The unlocked phase produced ten non-PASS outcomes in each set. In the original set, B10 printed trap diagnostics before the host timeout and is counted once as TIMEOUT. Some multicore output is interleaved.

The observations support the importance of allocator synchronization for this revision and workload. A PASS is one observed successful invocation, not proof that no harmful schedule exists. A timeout does not prove deadlock. The test may miss lost pages or corruption outside its user buffers. The elapsed times include output collection and are not performance benchmarks.

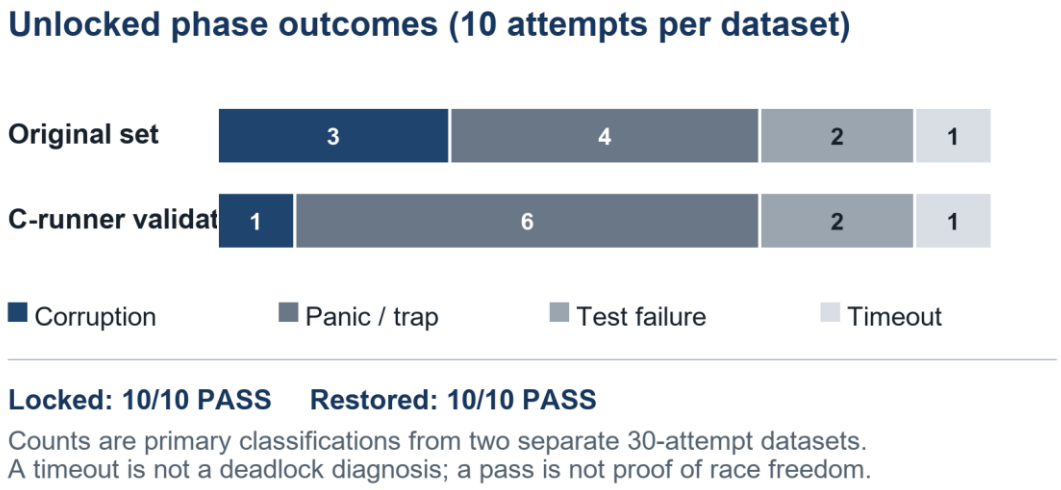


Figure 4. Comparison of primary outcomes across the two separate datasets.

9.1 Original observations: all 30 attempts

The following table reproduces every primary outcome and log filename from evidence/results.csv. The original unlocked set contains three corruption records, four panic/trap records, two test failures, and one timeout.

Table 15. Original dataset, attempt-level results

Run ID	Phase	Primary outcome	Elapsed (s)	Raw log
A01	locked	PASS	0.646	evidence/A01-locked.txt
A02	locked	PASS	0.65	evidence/A02-locked.txt
A03	locked	PASS	0.647	evidence/A03-locked.txt
A04	locked	PASS	0.642	evidence/A04-locked.txt
A05	locked	PASS	0.655	evidence/A05-locked.txt
A06	locked	PASS	0.639	evidence/A06-locked.txt
A07	locked	PASS	0.634	evidence/A07-locked.txt

Run ID	Phase	Primary outcome	Elapsed (s)	Raw log
A08	locked	PASS	0.645	evidence/A08-locked.txt
A09	locked	PASS	0.643	evidence/A09-locked.txt
A10	locked	PASS	0.654	evidence/A10-locked.txt
B01	unlocked	PANIC OR TRAP	0.875	evidence/B01-unlocked.txt
B02	unlocked	CORRUPTION	0.891	evidence/B02-unlocked.txt
B03	unlocked	PANIC_OR_TRAP	0.899	evidence/B03-unlocked.txt
B04	unlocked	PANIC_OR_TRAP	0.879	evidence/B04-unlocked.txt
B05	unlocked	CORRUPTION	0.906	evidence/B05-unlocked.txt
B06	unlocked	CORRUPTION	0.902	evidence/B06-unlocked.txt
B07	unlocked	TEST_FAILURE	0.947	evidence/B07-unlocked.txt
B08	unlocked	TEST_FAILURE	0.945	evidence/B08-unlocked.txt
B09	unlocked	PANIC OR TRAP	0.877	evidence/B09-unlocked.txt
B10	unlocked	TIMEOUT	60.137	evidence/B10-unlocked.txt
C01	restored	PASS	0.648	evidence/C01-restored.txt
C02	restored	PASS	0.651	evidence/C02-restored.txt
C03	restored	PASS	0.661	evidence/C03-restored.txt
C04	restored	PASS	0.648	evidence/C04-restored.txt
C05	restored	PASS	0.64	evidence/C05-restored.txt
C06	restored	PASS	0.636	evidence/C06-restored.txt
C07	restored	PASS	0.652	evidence/C07-restored.txt
C08	restored	PASS	0.645	evidence/C08-restored.txt
C09	restored	PASS	0.647	evidence/C09-restored.txt
C10	restored	PASS	0.658	evidence/C10-restored.txt

9.2 C-runner validation: all 30 attempts

This independent table reproduces c_validation/results.csv. Its unlocked phase contains one corruption record, six panic/trap records, two test failures, and one timeout.

Table 16. C-runner dataset, attempt-level results

Run ID	Phase	Primary outcome	Elapsed (s)	Raw log
A01	locked	PASS	0.857	c_validation/A01-locked.txt
A02	locked	PASS	0.848	c validation/A02-locked.txt
A03	locked	PASS	0.847	c_validation/A03-locked.txt
A04	locked	PASS	0.864	c_validation/A04-locked.txt
A05	locked	PASS	0.843	c_validation/A05-locked.txt

Run ID	Phase	Primary outcome	Elapsed (s)	Raw log
A06	locked	PASS	0.862	c_validation/A06-locked.txt
A07	locked	PASS	0.841	c_validation/A07-locked.txt
A08	locked	PASS	0.846	c_validation/A08-locked.txt
A09	locked	PASS	0.855	c_validation/A09-locked.txt
A10	locked	PASS	0.852	c_validation/A10-locked.txt
B01	unlocked	PANIC_OR_TRAP	0.780	c_validation/B01-unlocked.txt
B02	unlocked	PANIC_OR_TRAP	0.779	c_validation/B02-unlocked.txt
B03	unlocked	PANIC_OR_TRAP	0.780	c_validation/B03-unlocked.txt
B04	unlocked	PANIC_OR_TRAP	0.778	c_validation/B04-unlocked.txt
B05	unlocked	PANIC_OR_TRAP	0.777	c_validation/B05-unlocked.txt
B06	unlocked	CORRUPTION	0.792	c_validation/B06-unlocked.txt
B07	unlocked	TEST_FAILURE	0.852	c_validation/B07-unlocked.txt
B08	unlocked	TIMEOUT	60.028	c_validation/B08-unlocked.txt
B09	unlocked	PANIC_OR_TRAP	0.762	c_validation/B09-unlocked.txt
B10	unlocked	TEST_FAILURE	0.865	c_validation/B10-unlocked.txt
C01	restored	PASS	0.876	c_validation/C01-restored.txt
C02	restored	PASS	0.852	c_validation/C02-restored.txt
C03	restored	PASS	0.864	c_validation/C03-restored.txt
C04	restored	PASS	0.886	c_validation/C04-restored.txt
C05	restored	PASS	0.872	c_validation/C05-restored.txt
C06	restored	PASS	0.868	c_validation/C06-restored.txt
C07	restored	PASS	0.876	c_validation/C07-restored.txt
C08	restored	PASS	0.889	c_validation/C08-restored.txt
C09	restored	PASS	0.867	c_validation/C09-restored.txt
C10	restored	PASS	0.859	c_validation/C10-restored.txt

The original set and C-runner set used the same xv6 revision, CPUs, RAM, guest test parameters, and phase counts. The C-runner helper changed how the host experiment was automated. The distinct unlocked outcome counts show why the two evidence sets must remain identifiable.

10. Conclusion, applications and future scope

The case study connects race conditions to a shared kernel invariant. xv6's kmem.lock serializes changes to the physical-page free list in kalloc() and kfree(). With the lock present, both recorded datasets passed all locked attempts. After the four allocator lock calls were temporarily removed, both sets recorded failures in all unlocked attempts. Restoring the allocator produced ten PASS outcomes in each restored phase.

The results demonstrate the practical purpose of mutual exclusion under the specified QEMU workload. They do not prove correctness for every schedule or quantify the exact cause of each panic or corrupted byte. The test does not check every possible allocator invariant, and the removal also changes local interrupt-handling behavior associated with the spinlock calls.

The main learning is that protection must cover the full read and update sequence that preserves an invariant. The same reasoning applies to kernel queues, process tables, buffer caches, counters, and shared application state. The inventory in this report is source-level analysis except for the physical allocator.

Future work could vary the CPU count and stress parameters, add allocator-specific checks that detect lost or duplicate physical pages, examine another shared structure under a separate protocol, and measure lock contention. Those experiments were not performed here.

References

- xv6 library (MIT PDOS): https://github.com/mit-pdos/xv6-riscv
- MIT xv6 RISC-V book — Locking chapter: https://mit-pdos.github.io/xv6-riscv-book/lock.html
- The Open Group Base Specifications — memory synchronization: https://pubs.opengroup.org/onlinepubs/9699919799/basedefs/V1_chap04.html
- Michael and Scott — Nonblocking algorithms and the ABA problem: https://www.cs.rochester.edu/u/scott/papers/1998_JPDC_nonblocking.pdf

Evidence note: the terminal figures are static renders reconstructed from saved .cast recording output; they are not live desktop screenshots, and this report and deck contain no video. The recorded transcripts and results.csv remain in the supplied evidence directories.